CS3213 Foundations of S. Engineering (FSE)

AY24/25 Sem 2. github.com/gerteck

- Software development processes (agile vs. plan-driven)
- Requirements engineering, Modeling, Software architectures
- Software testing, Static analysis, Debugging, fault localization.

1. Software Engineering

Concerned with all aspects of software production, initial conception to operation and maintenance. Team development of multiversion programs. People at core, code important but only small part of building product.

- **Software engineering**: *practicalities* of developing, delivering useful software. Programming integrated over time. Software sustainability to keep software evolvable over time. People as first-order success driver.
- **Origins**: 1965s software crisis, Margaret Hamilton coins software eng. Bugs cost - 2022 US $2.41 trillion, technical debt biggest obstacle to existing codebases. E.g. crowdstrike 2024 outage, DBS 2023, Java Log4J 2021 zero-day vulnerability. Fred B. – no silver bullet (M. Man-Month).
- **Complexity**: *Essential complexity* - inherent, e.g. work w. stakeholders to determine product. *Accidental complexity* - solvable, e.g. buffer overflow exploitation protection. Hardest single part – deciding what to build.
- **Techniques to address key challenges**: Address users' needs with software, correctness and reliable, efficiency with limited resources.

Central Software Engineering Activities

Software Specification: Define functionality, constraints on operation.
Software Development: Implement to meet specification.
Software Validation: Ensure performance & requirements met.
Software Evolution: Adapts the software to evolving customer needs.

"Shifting Left" - more costly to address a defect late in the process. SE processes have been "shifting left".

SWE Qualities

- **Personal Pinciples**: Humility (Recognize limitations, self-improvement), Respect, Trust. Thrive in ambiguity, value feedback, challenge status quo, users first. Improving, Passionate, Knowledge about People/Org, Sees Forest & Trees, Create shared context, Creates safe haven. Working alone increases risks, design mistakes, irrelevance, and "bus factor."
- **Software Product Attributes**: *Elegant*: Simple, intuitive designs: easy to understand, maintain. *Anticipates Needs and future requirements*.

2. Software Engineering Processes

Software process model: rep. of software process (sequence of activities). Illustrates different approaches to developing software.

- **Waterfall Model**: Simple, linear plan-driven software process model. Output of one phase feeds into the next. Development starts only after req. eng and design. **Pros/Use Cases**: Embedded-hardware /Safety-critical /Large systems. Works well if requirements clear, large projects, or distributed teams. **Cons**: accommodate change difficult.
- **Incremental Development**: Overlapping phases of specification, development, and validation. Built in increments, adding functionality. Can be plan-driven, agile, or mix. **Pros**: Lower cost of accommodating change, faster delivery. **Cons**: Harder to measure progress, may lead to system degradation (require refactoring).
- **Integration and Configuration / Reuse-based**: Reuse existing components/framework/library over from scratch. May need redefine reqs. based on identified components. **Pros**: Reduce amt. to develop, lower cost, risk. **Cons**: Limit customizatn, third-party dependency, integration complexity.

Agile Software Engineering

Agile frameworks (e.g. scrum) not specific to software dev. Designed for changing, unclear requirements and incremental dev, deliver value quickly.

- **Agile**: Both mindset and concrete mngmt engineering framework. Focus on deliver value, avoid waste. Feedback loops, customer involvement. Adaptive to change. Lightweight process, empower people, lean manufacturing, buzzword certifications.
- **Historical Development and Origins**: 1980-90s: Trad. planning/dev processes. Late 1990s: Dissatisfaction, agile as alternative to rigid waterfall. 2001: Agile Manifesto by reps from frameworks like Scrum, XP, Feature-Driven Development (FDD). Influenced by *lean manufacturing*, *The New New Product Development Game (1986)* – overlapping stages, built in instability, subtle control, self-organizing teams (autonomy, self-transcendence of teams, cross-fertilization, learning & learning transfer cross teams).
- **Lean software development**: Limit waste (hardship in daily work) – partially done work, non-value adding processes/features, task-switching, waiting for input, defects, handoff motion.
- **Manifesto Principles**: Individuals & interactions over processes. Working software over compreh. docs. deliver frequently. Customer collab over contract nego (comms daily), Respond to change over following plan (wlc. changing req.). Working software as progress measure, sustainable development. technical excellence and good design enhances agility, simplicity is essential. regular reflection.
- **Agile Adoption** : Larger the org, more likely to use hybrid mode. Bigger teams likely still use waterfall. Agile has concrete benefits - better collaboration, alignment to business needs, better quality software. **Problems**: business side slow to embrace. Too many mixed systems, siloed teams delays. Incorrect practice, managment buy-in, pain of lacking documentn.

Scrum

'Agile synonym'. General, lightweight framework for project management.

- **Core Principles**: Empirical process control and lean thinking. Focuses on delivering value through increments in sprints.
- **Scrum Pillars**: Transparency (work and process visibility), Inspection (frequent evaluation), and Adaptation (adjustments as needed).
- **Scrum Team**: Self-managing, no hierarchies, typically $\leq$ 10members. *Scrum Master*: Facilitates Scrum process, removes impediments. *Product Owner*: Maximizes product value, manages Product Backlog. *Developers*: Responsible for creating Increment each Sprint.
- **Scrum Events**: *Sprint*: Timeboxed iteration ($\leq$ 1 month), deliver working Increment. *Sprint Planning*: Defines Sprint Goal, selects backlog items. *Daily Scrum*: 15-minute daily sync to track progress and adjust plan. *Sprint Review*: Team present results, adjust Product Backlog. *Sprint Retrospective*: Reflect on improvements for future Sprints.
- **Scrum Artifacts**: *Product Backlog*: Ordered list of work items aligned with Product Goal. *Sprint Backlog*: Selected items for Sprint w. actionable plan. *Increment*: Deliverable output meeting defn. of Done.
- **Timeboxing**: Sprint(S): $\leq$ 1 mnth. S. Planning: $\leq$ 8 hrs. Daily Scrum: 15 minutes. S. Review: $\leq$ 4 hrs. S. Retrospective: $\leq$ 3 hrs.
- **Kanban Board**: Often used in Scrum to visualize workflow. It helps teams track progress, limit work in progress (WIP), and improve flow (movement of potential value through a system).
- **Definition of Workflow (DoW)**: A shared understanding of how work items flow through the system. Includes *Units of value* (work items or user stories), defined states (stages) work items flow through, policies for how items move through each state. WIP limit controls enforced.

Extreme Programming (XP)

Highly influential agile methodology (late 1990s). "extreme" approach to iterative development and best practices. New ver may built several times/day, increments every 2 wks, All tests successful for accepted builds.

- **Pair Programming**: One dev programming, other review, focus on big picture. (driver & navigator). Both can think and interact at same level of interaction. If one greater expertise likely to dominate interaction. Switch roles after a time period, and switch partners frequently. *Pros vs Cons*: Fewer bugs, better code understanding, peer learning, but lower cost / scheduling efficiency, personal clash.
- **Collective Ownership**: Everyone responsible for all code, anyone can change any code. Pair prog facilitates this, as everyone better understands larger system, contribute to different parts of code.
- **Test-driven development**: Unit tests written before code, test all potential fail cases. helps development, facilitates other CI, refactoring etc.
- **Continuous Integration**: Dev team work on latest ver. Integrate code to main branch freq. Facilitate using unit tests. (GH Actions, Jenkins).
- **Simple Design, Refactoring**: "simple is best", refactoring as part of the lifecycle, keep it maintainable, easier to extend.
- Others: on-site customer for access system requirements, sustainable pace for net effect on code quality / long term productivity.

DevOps

In incremental development, challenge software deployment quickly & reliably. Involves collaboration btw. dev (scrum teams) & operations teams.

- **DORA (DevOps Research and Assessment)**: Framework for measuring and improving DevOps performance through key metrics like deployment frequency, lead time for changes, change failure rates, mean time to recovery. *Observability and Monitoring* for measuring DevOps performance. Monitoring of running code and infrastructure provides insights to optimize the value stream.
- **Silos**: Traditionally, dev and ops teams separated, conflicting goals. Developers focus on delivering features quickly, while operations prioritize stability and uptime. DevOps bridges gap by fostering collaboration.
- **DevOps Overview**: Unified approach, emphasize automation, collaboration, continuous delivery. Popularized 2010, aims for routine, predictable deployments. *DevOps vs. Agile*: Agile focus on iterative dev, deliver value, DevOps extends by automating deployment pipeline. Emphasis on CI, delivery, feedback loops. "Infinite loop" of dev and ops.
- **DevOps Principles**: *Flow*: Reduce batch sizes & handoffs (work passing across depts/teams) to streamline delivery. *Feedback*: Fast feedback loops to detect, resolve issues early. *Continuous Learning*: Promote experimentation, learning, mechanisms to share knowledge. Safety culture.
- **Learning from Incidents**: Encourages *blameless postmortems* to investigate failures, implement preventive measures. Fosters culture of experimentation and learning.
- **Value Stream Mapping**: Technique to visualize and analyze the flow of work, identifying inefficiencies and improving delivery processes. (E.g. develop unit test $\rightarrow$ code solution $\rightarrow$ PR reviews $\rightarrow$ QA $\rightarrow$ regression, performance test $\rightarrow$ deploy $\rightarrow$ smoke test $\rightarrow$ UAT test)
- **Continuous Integration and Deployment (CI/CD)**: Automates the integration, testing, and deployment of code changes, enabling frequent and reliable releases (push to production). *Infrastructure as Code*: Treats infrastructure configuration like code, using version control, automation to manage servers and environments.

3. Software Requirements Engineering

Functional vs. Non func. Req. Quality Attribute Scenarios. Elicitation & Analysis. Documentation. Validation. Process Models. Change Mngmt.

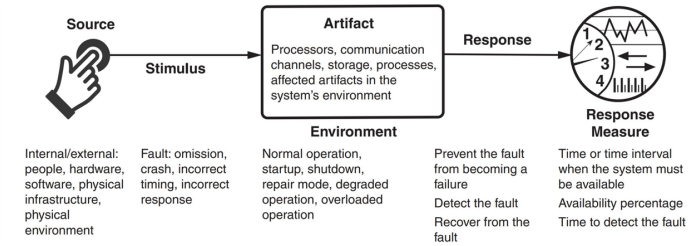
- **Requirements** describe services system should provide, constraints on its operation. Reflect customer needs for systems w certain purpose e.g. device control, order placement, or information retrieval.
- **Requirements Engineering (RE)**: Process of discovering, analyzing, documenting, and validating these services and constraints. RE focus on problem space, not solution space.
- **Wicked Problem**: No definitive formulation or stopping rule, solutions not true/false but better/worse, no immediate/ultimate test for solutions. Every solution is a "one-shot operation" with significant consequences. Each problem is unique, often a symptom of another problem. No exhaustive set of potential solutions or permissible operations. Social planners are accountable for the consequences of their actions.

Requirement Engineering (RE), Why & Challenges

- **High Cost of Errors**: Errors during requirements phase costly.
- **Importance of RE**: Facilitates communication between stakeholders. (*Customer Relations and Communication*). Provides the foundation for defining structure, behavior of software system. Serves as basis for testing, accepting final product. (*QA, Acceptance*) Dictates quality of system, risks, and operational overhead. (*Maintenance and Evolution*)
- **RE Challenges**: Difficult to determine if all relevant requirements are collected. Requirements engineering is resource-intensive, often one-shot effort. Requirements cannot be generalized.
  - *User-Related*: conflicted views, incomplete understandg of needs, limits.
  - *Analyst-Related*: Poor problem domain knowledge, omit "obvious" info.
  - *Communication*: Requirements evolve w. time, Ill-defined system boundary. Vague, untestable requirements ("user-friendly," "robust").
- *NaPiRE* (RE pains): Incomplete/hidden req, comm. flaws, time-boxing (not enough time), moving targets, inconsistent/underspecified req...

Functional vs. Non-Functional Requirements

- **Functional Requirements**: Describe what system should do. Stakeholders typically focus on functionality, not determine system architecture. (any functionality implementable using various designs.)
- **Non-functional Requirements**: Define system qualities, not specific services (e.g., performance, security, usability). Often more critical than FRs, may involve implicit stakeholder expectations. Can conflict (e.g., security vs. usability). Influence architectural tradeoffs (e.g., *quality attributes*).
- Non-functional requirements should be quantitative to be testable. E.g. ("throughput suff. high." vs. "throughput to exceed 1,000 query/sec")
- **Quality Attribute Scenarios**: framework to clarify, document NFRs. Up to 6 elements: (*Stimulus*: An event arriving at the system. *Source*: Origin of stimulus (e.g., human, system), *Artifact*: part of system affected by stimulus, *Response*: system's reaction. *Response Measure*: Quantifiable metrics for response. *Environment*: scenario context).



Other Non-Functional Requirements and Metrics

- **Usability**: How easily users accomplish tasks, support provided. *Metrics*: Task time, errors no., learning time, satisfaction.
- **Availability**: System readiness to perform tasks, including reliability and recoverability. *Metrics*: Availability %, time to detect/repair faults, etc.
- **Performance**: System responsiveness and throughput. *Metrics*: Latency, throughput, jitter, resource usage, etc.
- **Modifiability**: The cost and effort of making changes to the system. *Metrics*: Effort, elapsed time, new defects introduced, etc.
- **Deployability**: The ease of deploying and rolling back system changes. *Metrics*: Cost, percentage of failed deployments, cycle time, etc.
- **Energy Efficiency**: System efficiency in energy usage (e.g IoT, AI).
- **Security**: Protecting data and systems from unauthorized access. (CIA Triad) *Metrics*: Accuracy of attack detection, size of trusted execution
- **Data Security and Privacy**: PDPA (Singapore), GDPR (EU).
- **Safety**: Avoiding system states that cause harm or damage. *Metrics*: Percentage of unsafe states avoided, recovery time, etc.

Stakeholders and Requirements Engineering (RE) Activities

RE processes vary based on the organization and project type.
3 Cs of user stories map to key activities:

Elicitation and analysis → Conversation (w. client).
Specification → Card (description).
Validation → Confirmation (acceptance tests to determine completeness).

Agile vs. plan-driven approaches overlap in elicitation but differ in emphasis: Agile focus on iterative feedback, collaboration. Plan-driven emphasize detailed specification and validation.

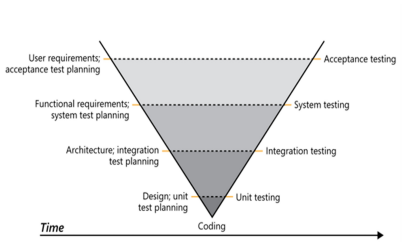
SRS: Software Requirements Specification: Central document, official statement of what should be implemented. May include user requirements, detailed specification of system requirements. Output of RE phase.

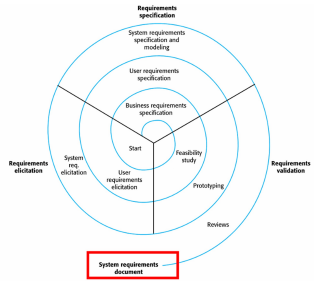
Stakeholders: person/org impacted / influencing system's req. *Identify stakeholders* as first step in RE, determines whose needs /desires addressed. Start with "baseline" stakeholders (e.g., users, decision-makers). Explore network to identify add. stakeholders.

- **Elicitation and Analysis**: Work w. stakeholders to get requirements. Elicitation techniques: (Closed/Open Interviews, Existing Docs Analysis – prior research reduces interaction time, (user-story) workshops – multiple stakeholders & resource intensive, FGs (users), prototyping, observe – discover implicit req. reflecting actual ways, personas). Supports user stories.
- **Specification**: Convert req. into standardized form (user-centric view).
  User Stories: represent. customer requirements (<As 'usertype', want 'goal', to 'reason'.>) Three Cs - card, convo, confirm. Epics – too large user stories to be split later. Include notes, NFRs (constraints).
  Use Cases: interaction btwn actor & system (e.g. check in flight). UML → use case diagram. May encompass multiple scenarios (single instance of usage e.g normal flow, crash, others etc). Natural Language – use standard format, consistency, associate rationales, avoid jargon.
- **Validation**: Ensuring requirements accurately define the desired system. (Agile: freq comm. w stakeholder, Plan: unclarities in req. document). Techniques: *Informal Reviews*: Peer review, colleague reviews requirements. Informal feedback collection (e.g., deskchecks, passarounds, walkthroughs). *Formal Reviews/Inspections*: Systematic analysis by a team of reviewers, roles: Moderator, reader, inspectors. Techniques: Defect checklists, requirement-by-requirement reviews.
- **Requirement document checks**: *Validity*: reflect real user current needs. *Consistency*: Avoid contradictory req. *Completeness*. *Realism*: feasible, within budget, tech constraints. *Verifiability*: testable.

Other Requirement Engineering Validations

- **Feasibility Study**: short study to assess: alignment with organizational objectives, budget and schedule, Integration with existing systems.
- **Prototyping**: Develop quick version of the system to validate requirements and design decisions. Reduces post-delivery changes by allowing users to experiment early.
- **Use Cases vs User Stories** (Use Cases – Plan-Driven): Specify functional requirements and serve as test bases. (User Stories – Agile): Act as placeholders for implementation, elaborated into acceptance tests.
- **Requirements as a Basis for Testing**: Requirements should be testable and used to derive test cases. User stories, cases as foundations for acceptance tests.
- **V-Model and Testing**: In plan-driven approaches, requirements validation aligns with testing phases in the V-Model.





Requirement Engineering Process Models

Agile Approaches: Agile avoids detailed requirements documents due to rapid changes. Relies on user stories and frequent stakeholder feedback. Short supporting documents may still define business and dependability requirements.
Plan-Driven Approaches: Follow a structured process model. RE phase is completed before implementation. Produces a requirements document, often part of the development contract.

- **User vs. System Requirements**: UR: High-level, natural language descriptions of system services and constraints. SR: Detailed descriptions of functions, services, and operational constraints.
- **RE Spiral Model**: iterative, plan-driven RE process model. Expands requirements incrementally, focusing on high-level business needs early and detailed system requirements later. Outputs SRS. Can incorporate agile development in later iterations.
- **RE Elicitation and Analysis Model**:
  Requirements Discovery (Interact w. stakeholders to uncover needs) → *Classification and Organization* (Group req. to coherent clusters) → *Prioritization and Negotiation* (Resolve conflicts, prioritize req) → *Documentation* (Record requirements for next iteration) → (repeat)
- **Requirements Change Management**:
  Established Plan for to manage evolving requirements.
  Unique identification of requirements (to cross-reference).
  Established change management process to assess impact and cost.
  Traceability policies to link requirements to design.
  Tool support for managing requirements

4. System Modeling and Software Architecture

Abstractions. Modeling Perspectives. UML. C4. Different Architecture Structures, Drivers, Patterns, Tactics.

Systems Modeling

Develop abstract models of a system, usually some graphical notation based on diagram types in Unified Modeling Language (UML).
Means to an end (developing software).

- Emphasized in plan-driven development, also used in agile in lightweight manner ("just enough" design).
- Formal models (mathematical) can be developed for auto verification.
- Models are created during requirements eng, system design, and post-implementation documentation.
- **Abstraction vs Representation**: Abstraction simplifies systems, omits details. Representation maintains all system information.
- **Different perspectives**: External, Interaction, Structural, Behavioral.

Modeling Languages

Unified Modeling Language (UML, most known, rich tool support, stand. notn), C4, BPMN, SysML, etc.

UML Key Diagrams:

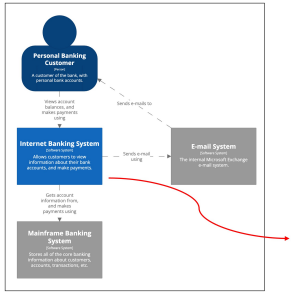
- Class Diagrams: Show object classes and associations.
- Sequence Diagrams: Show interactions btw. actors & components.
- Activity Diagrams: Show process or data flow activities.
- State Diagrams: Show system reactions to events.
- Use Case Diagrams: Show interactions btwn system & environm.

C4 Model: Agile Architectural Modeling. For software architectures. Do "just enough" design / upfront thinking for starting point/ general direction.

- Hierarchical architectural modeling (4Cs): System Context, Containers, Components, Code. Notation Independent.

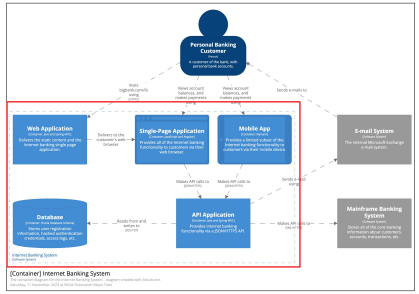
- *System Context diagram* → starting point, how software fits into world.
- *Container diagram* → software system, high level building blocks.
- *Component diagram* → individual container, decomposes containers to identify major structural building blocks (i.e., components). Not recommended unless level of detail adds value.
- *Code diagram* (e.g. UML class) → indivdual component implementation. Not recommended for C4 / Agile context use.

Systems Context Diagram



Big picture of the system landscape, focusing on people and software systems

Container Diagram



A container diagram zooms into the system boundary. Container is separately runnable/deployable component e.g. web app, mobile app, database or file schema

Software Architecture

The overall structure of a system, including components, relationships, and properties. How devs explain system at high level.

- S. Architecture vs S. Design: Software Architecture is part of design, focusing on high-level structures. Design (e.g. detailed class design)
- Influences quality attributes, modifiability, and communication among stakeholders. Mitigate highest priority risks. Difficult to change later.
- **Architectural Drivers**: Architecturally Significant Requirements (ASRs): Requirements that significantly impact architecture. Non-functional req. (e.g., availability, performance) are often ASRs. Functionality must have major influence on architecture. (E.g. the system should provide five nines (99.999 percent) availability).

Often, system architec. modeled informally w. simple block diagrams.

Architectural Reasons and Consequences

- **Quality Attributes**: Architecture significantly influences whether system meets quality attributes (non-functional requirements). (E.g. *High Performance* – require managing time-based behavior and shared resource access, *Safety and Security*...)
- **Modifiability and Change**: Modifiability is a critical quality attribute. Requires assigning responsibilities to components to limit the impact of changes. *Categories of changes*: Local – affect only single component, Non-local – Affect multiple components (e.g. UI). Architectural Change – Affects entire system (e.g., single-thread to multi-thread). Good architecture ensures most changes local or non-local.
- **Communication Among Stakeholders**: Architecture as common abstraction for mutual understanding, negotiation, and consensus.
- **Additional**: Enables reasoning of cost, schedule, basis for reusable models in product lines, focuses on component assembly, channels developer creativity by restricting design alternatives, foundation for training new team members, enable early prediction of system qualities, defines constraints for subsequent implementation, influences org structure (Conway's Law), supports incremental dev.
- **Conway's Law**: Organizations design systems that mirror their communication structures – Arch. often forms basis for work-breakdown structures.

Architecture Patterns and Tatics

Design is complex activity, architecture is a tradeoff of attributes.
Architectural tactic: design decision that influences a quality attribute.

- **Attribute-Driven Design (ADD)**: Systematic approach to architecture design. Each design iteration, address an ASR.
  Steps: Identify ASRs → establish iteration goals (satisfy set of ASRs) → refine structure → select designs → instantiate patterns → record decisions → analyze designs.

Architectural Patterns *Established architectural solution, may comprise multiple architectural tactics.* Often found useful over time/domains, documented and disseminated.

- **Antipattern – Big Ball of Mud**: Common in practice. Lack of architectural design, Business pressure, Erosion of architecture over time. Poor maintainability, extensibility (Quality Attributes)

Common Architectural Patterns

- **Layered Architecture**: Structures systems into layers with defined interfaces. Layer only uses layer directly beneath, through public interface. (constraints may be violated e.g. for performance, layer bridging, upper layer use deeper lower layer).
  Pros: portability (to diff env/OS), modifiability (lower layers changeable under hood, less interfaces to understand), reuse
  Cons: Layering possible performance penalty. If not properly designed, can impede (missing nec. low level abstraction needed at higher level).

- **Pipe-and-Filter**: Processes data through independent filters connected by pipes. *Filters*: processing components, take input, produce output, no assumption on other filters, stateless. *Pipes*: connect multiple filters with each other. (E.g. MailMan - add header, digest, archive, outgoing).
  Pros: modifiability (independent filters) and reconfigurability, evolution (add/combine filers differently)
  Cons: format data transfer must be set. performance as every filter must parse input convert to output format.

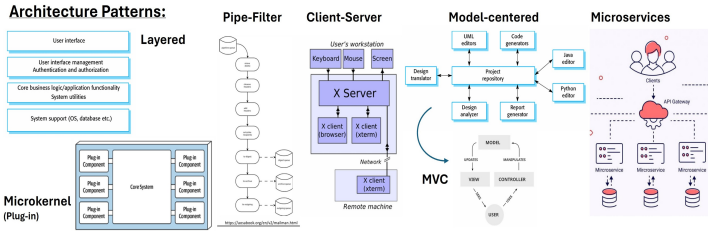
- **Model-Centered**: Independent components interact with a central model than each other. Repository style. (E.g. IDEs).
  Pros: components can be independent. highly modifiable. concurrency. State/data manged consistently (in one place).
  Cons: model/repo is single point of failure. change to model = many changes. Distributing repo can be difficult.

  MVC design pattern: "instantiation" of model-centered, goal of MVC to separate model (business logic) from UI (controller/view).

- **Microkernel (Plug-in)**: Base system with plug-in components (e.g., Eclipse SDK, web browsers).
  Pros: modifiability, extensibility, testability. Plugins can be developed, tested, evolved by diff. teams independently.
  Cons: can introduce security /privacy vulnerabilities

- **Client-Server**: central server, provide service to multiple distributed clients. Client request service, server cannot initiate. (E.g. web server) *Discovery*: client use discovery service, server respond w. agreed protocol. *Interaction*: client sends request, server process, respond.
  Pros: Low coupling btwn server clients, no coupling among clients. Easily scale no. clients and server. Both can evolve indptly.

- **Microservices**: Independently deployable services communicating via APIs. Typically stateless, small teams, small code bases. Service dependencies acyclic. Popularized by large companies. Goal - decoupling, microservices expected to be independent w own domain/workflow. typically smaller than services in SOA.
  Pros: quick time to market, indiv. deployability, test, scalability.
  Cons: network comm. overhead, complex transactions, challenge in design /maintain sea of microservices

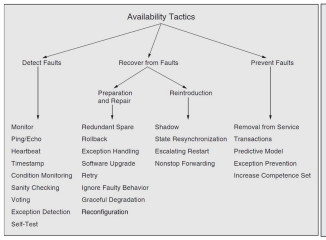


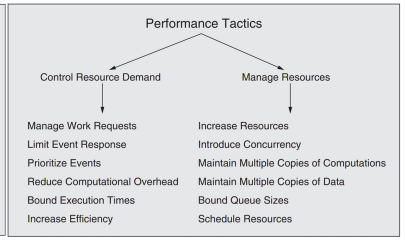
Monoliths vs. Service-Oriented Architecture (SOA): Monolith typically single deployable unit (diff. way of partition e.g. layered archi.). SOA for distributed-architecture styles, services separately deployed, only interact over interfaces, network. Related to microservices.

Architectural Tactics

Assign decision that influences a quality attribute. Tatics are design decisions that influence quality attributes. (modify existing patterns) E.g.

- *Availability*: Detect faults (monitoring, heartbeat), recover (redundant spare, roll-back), prevent faults (ACID transactions, remove from service).
- *Performance*: Manage work requests, limit event response, maintain multiple copies of data, schedule resources.





"20 minutes rule": Continuous learning essential for staying updated.

5. Software Testing

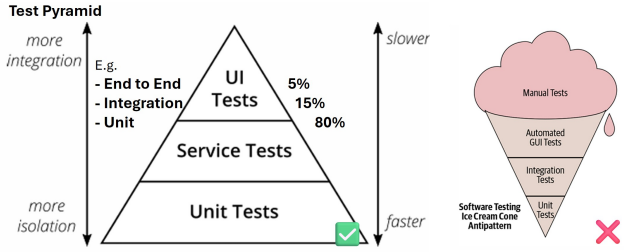
Test levels. Flaky tests. Specification-based/Structural/Mutation testing.

Purpose of Testing: *Defect Testing*: Find bugs in software. (Tests show presence, but never absence.) *Validation Testing*: Ensure system meets user requirements.

- **Verification vs. Validation**: *Verify*: Build product right, conforms to specification. *Validate*: Build right product, software meets user needs.
- **Test Case**: A test case consists of: *Test Input*: Arguments, system state, or actions, *Test Oracle*: Mechanism, determine test pass or fail.
- **Test Oracle**: Can check functional properties (e.g., correctness) or non-functional properties (e.g., performance, security).
- **Manual vs. Automated Testing**: *Automated*: Use tools to auto generate inputs and apply oracles (e.g., JUnit). *Manual*: Tester manually inputs test cases and acts as oracle. ('Traditional' - auto as in test automation).
- **Pesticide Paradox**: 'Every method used to prevent/find bugs leaves residue of subtler bugs which those methods ineffectual'. Diff testing methods have pros, cons. Use many, similar to pests build immunity. (Law of diminishing return).

Testing Levels

- **Unit Tests**: Test indiv. components (e.g., single method or class, e.g. testing a method converting AST to string). *Pros*: Fast execution, easy control & write (no additional setup of (e.g., no DB, web services).
  Cons: Lack realism, don't represent real system execution. May miss integration bugs, often require mocks to simulate ext. dependencies.
- **Integration Tests**: Test interactions btw. multiple compnt. Focus on integrating target cmpnt. w. external cmpnt. (e.g., ext DB, web service). (e.g. SQLancer test query plan converted to expected internal repnstn.)
- **System Tests**: Test entire system as whole. *Pro*: realistic test env. *Cons*: slower execution, harder to write (complex setup), prone to flakiness (non-deterministic results). (e.g: Ensuring SQLancer compatibility with database system.)
- **Test Flakiness**: Pass/fail non-deterministically. Low result confidence.
  Causes: Concurrency issue (e.g., race conditions), async waits (e.g., not properly waiting for server responses), test order dependencies (e.g., improper cleanup), test timeouts, resource leaks. (e.g, test fail from `ConcurrentModificationException` in multi-threaded code.)
- **Test Pyramid**: strategy for balancing test granularity. Many small, fast unit tests. Fewer integration, system tests. *Antipatterns*: Too many high-level tests, slow execution, maintenance challenges.



- **Test Sizes at Google**: Tests classified by size to for efficiency, determinism.
  Small: Single process/thread, no I/O, fast, deterministic.
  Medium: Multiple processes, allow localhost network calls.
  Large: No restrictions, slower, less deterministic.

Testing and Processes

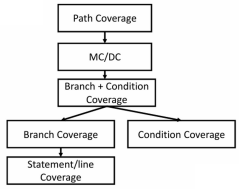
- **V-Model**: extension of waterfall model. Testing is planned alongside RE, archi, design phases. (E.g. acceptance test derived during RE).
- **User Accept Testing**: validate system meets user req, w customer participation. In *plan driven*, explicit phase after system implementation. In *agile*, UAT is interleaved (e.g., acceptance criteria in user stories).
- **Test-Driven Development (TDD)**: Emphasized by XP. Use as design process, focus 'happy path'. *Pros*: Focus on req, provide quick feedback. Encourages testable code, allows incremental exploration.
  Steps: Write test → ensure fails → simplest code to pass → refactor.
  When to Use: Useful for complex prob, unclear soln (incremental experimentation). Less useful if solution already clear.
  Schools of thought: (Classicist/Detroit, Inside-out start e.g. business rules → UI). (London/Mockist Outside-In: Start outside, use mocks).
- **Black-box vs. White-box Testing**: *Black-box 'specification testing'*: No internal info, based on requirements or function documentation. *White-box 'structural testing'*: Internal info, focus code structure, logic.

Black-box Testing

- **Specification-based Testing**: Derive from requirements (e.g., user stories, use cases, docs). Functional & non-functional requirements.
  Testing Workflow: Understand requirements (inputs, outputs), explore program (functionality), identify partitions (eqv classes of input/output), analyze boundaries, devise and automate test cases, augment test suite with creativity, experience on common bugs.
- **Equivalence Partitioning**: Divides inputs into equivalent classes (e.g., null, empty, single-char string). Output partn may help spot input partn.
- **Boundary Value Analysis**: Tests edge cases (e.g., min/max).

White-box Testing

- **Structural Testing**: use source code to guide testing. Catch requirement independent implementation aspect (e.g. cache, DSA). Focus coverage criteria. (e.g. Jacoco basic Java line coverage.)
- **Coverage Criteria**: *Line Coverage* (percentage executed). *Branch, Condition Coverage*: Percentage of branches (if [true/false], fors, whiles), $(a||b)$ executed. Condition coverage does not imply branch coverage. *Path Coverage*: Strongest criterion. Often impossible (e.g. loop).
- **Modified Condition/Decision Coverage (MC/DC)**:
  Test impt combinaton of cond, less cost.= $\frac{\#\text{conditions independently affecting decisions}}{\text{total \#conditions}}$.
  Every condition in a decision (e.g., if $(A\&\&B)$) takes every possible outcome (T and F). Ensures conditions independently affect decision outcome. Final set of tests, less than needed for path coverage. ($n+1$ test cases common lower bound for n conditions).



MC/DC: Modified Condition/Decision Coverage

Test case	A	AND	B	result	applying MCDC rules
1	1		1	1	A and B are decisive for the result "1"
2	1		0	0	B is decisive for the result "0"
3	0		1	0	A is decisive for the result "0"
4	0		0	0	This test case can be skipped based on the MCDC-rules, as both A and B have been decisive for both a true and a false result of the complex condition

- **Mutation Testing (White-box)**: Evaluate test suite quality by introducing bugs (mutants) and checking if tests detect them. (test case should 'kill' mutant - fail the test case). % of bugs detected = quality of test suite.
  Steps: Select statement in code, mutate code, run test suite, record outcome. Undo change, repeat. Calculate mutation score: $\frac{\#\text{killed mutants}}{\#\text{mutants}} \times 100$.
  (e.g. Pitest: Mutation testing tool for Java.).
  Pros: Identify undertested code.
  Cons: Comptn expensive; equivalent mutants may not change behavior.

6. Debugging and Fault Localization

Scientific Method to Debugging (Systematic Approach). Statistical Fault Localization. Input Reduction. Isolating Bug inducing Changes.
Mainly, build mental model to approach debug from different angles, and some techniques useful as tools (e.g. reduction tools).

Debugging

Software developers spend 35–50% of time validating, debugging software. Est. cost of debug, test, verification up to 50–75% project budget. Identify systematic, effective approaches, enhance productivity.

- **Shifting Left**: Debug during software development (e.g., due to failing tests or static analysis reports), but also after deployment (user reports).
- **How NOT to approach debugging**: Can change code and try until works. But if don't understand underlying cause, may not fix bug's root cause.
- **Typical strategy to debug?**:
  - **Printf debugging**: Simple, intuitive, language tool agnostic. But can quickly be confusing, forget to remove print, may require recompilation.
  - **Logging**: More systematic alternative to print statements. Multiple debugging levels supported (E.g. INFO, WARN, ERROR, FATAL).
  - **Debugger**: Language-specific debugger. Set breakpoints, stepping in/over/out methods, stepwise execution, inspect program state (variables, stack trace.) Can be more systematic, display more info, set breakpoints properly. However, still require strategy with debuggers.
- Challenges: Program states very large, challenging to search all manually. Not clear if (intermediate) states correct or not. Execution could have millions of steps. Hence, need good debug strategy to focus search.

Fault vs. Defect vs. Failure:

- **Fault**: Location of flaw in code, causes incorrect behavior/unwanted state.
- **Defect (Bug)**: Manifestation of fault in software. (Error in program code)
- **Failure**: Deviation of behavior from expected output. (Externally visible)

Through unit test, observed a failure. In order to debug the failure, we need to trace it back to the defect that caused the first fault.

Scientific Method to Debugging

Systematic process of identifying faults. Basically, observe failure, generate hypotheses on cause, test hypotheses by experiment (e.g., add debug , assert statements). Refine or confirm hypotheses based on results.

1. Formulate a question.
2. Come up w. hypothesis based on knowledge obtained while formulating question, that may explain observed behavior.
3. Determine logical consequences hypothesis, formulate prediction that can support or refute hypothesis. Ideally, prediction would distinguish hypothesis from likely alternatives.
4. Test prediction (and thus hypothesis) in experiment. If prediction holds, confidence in the hypothesis increases; otherwise, it decreases.
5. Repeat Steps 2–4 until no discrepancies between hypothesis and predictions and/or observations.

- **Scientific process Theory**: a hypothesis that is consistent with earlier observations, predict future obsv (program behavior). $\rightarrow$ Diagnosis.
- **Diagnosis** (Debugging): explains both causality, (how failure came to be), and incorrectness (how to correct the code accordingly), then it is time to actually fix the code accordingly.

For complex issues, might have days/weeks to debug, systematic debugging strategy is useful. Also helps to systematically log and create a table etc.

Rubberducking: Explaining code line-by-line to an inanimate object (or another person) to identify logical errors or misunderstandings.

Program Slicing

Identifying parts of the program (slice) that influence/ are influenced by specific variable/ statement. Automatically determine which earlier variables influenced the current erroneous state.

- **Definition**: Slicing extracts a subset of program that potentially affects variables/ values at specific program location. A slice identifies the subset of the program that could have influenced the variable/returned value.
- **Tracking Origins**: Start with faulty state (failure), trace origins to earlier states. Identify if origins faulty or correct. Continue tracing till faulty state with only correct origins found, indicating the defect.
- **Advantages**: Narrows scope of debugging, rule out locations that cannot have effect on failure. Bring together possible origins that may be scattered across code. AKA helps consolidate scattered dependencies into a single, manageable view.
- **Types of Dependencies**: *Data Dependency*: Variable's value depends on prior variable. (e.g. fn params). *Control Dependency*: Execution of statement depends on condition. Can reason using program dependence graph.
- **Static vs. Dynamic Slicing**: *Static Slicing*: Considers all possible executions of the program. (e.g. include whole method, both if and else branch). *Dynamic Slicing*: Focuses on a specific execution path based on concrete inputs. (e.g. only consider if branch, as else branch not executed).
- **Forward vs. Backward Slicing**: *Backward Slicing*: Identifies what influenced a particular value. *Forward Slicing*: Identifies what statements are influenced by a particular value.

Slicing applications: As mental model, some support in IDEs, papers published, program understanding, maintenance, security analysis.

Statistical Fault Localization

Automatically localize faults using statistical reasoning.

- **Observation**: Program failures correlate with events that precede them. (Correlation does not imply causation, but can help find faults.)
- **Tarantula Algorithm**: Assigns suspiciousness scores to program elements based on involvement in failing and passing test cases. Hue $= [0, 1]$
  - Formula: [Suspiciousness(line) = $\frac{\frac{failed(line)}{totalfailed}}{\frac{failed(line)}{totalfailed} + \frac{passed(line)}{totalpassed}}$]
  - Visualize using color coding (red: suspicious, green: safe).
- **Limitations**: Provides no direct information about root cause beyond identifying suspicious lines. Effectiveness depends on complexity of bug. Limited tool support in IDE integrations. Research shows such approaches provide only limited support to experts.

Test-Case Reduction

Reduce large failure-inducing inputs to minimal reproducible test cases. E.g. Mozilla BugAThon to simplify bug reports.

- **Small Scope Hypothesis**: "A high proportion of errors can be found by testing a program for all test inputs within some small scope". Most bugs triggerable w. small, simple inputs. Can reduce the bug-inducing input.
- **Test-case reduction tools**: reduce input element, apply (user-supplied) test oracle, check whether the bug is still being triggered If so, continue applying another reduction. If not, undoes change, attempts another reduction. **E.g.**: Using C-Reduce to minimize SQL queries that expose bugs in SQLite.
- **Delta Debugging**:
  - Systematically removes elements from the input while ensuring the bug is still triggered. Behaves like binary search but explores combinations of smaller blocks when binary search fails. Iteratively increases granularity if neither partitions nor complements trigger the bug.
  - Simply: start reducing, consider granularity of n=2 (two halves). Consider partitions of input, and then complements (Whole input minus the specific partition). If any of partitions cause test to fail, continue reducing partition. Otherwise, reduce complement. If both do not work, increase granularity (create more partitions).

Minimizing Delta Debugging Algorithm

Let *test* and $c_{\text{✘}}$ be given such that $test(\emptyset) = \text{✔} \wedge test(c_{\text{✘}}) = \text{✘}$ hold.
The goal is to find $c'_{\text{✘}} = ddmin(c_{\text{✘}})$ such that $c'_{\text{✘}} \subseteq c_{\text{✘}}$, $test(c'_{\text{✘}}) = \text{✘}$, and $c'_{\text{✘}}$ is 1-minimal.
The *minimizing Delta Debugging algorithm* $ddmin(c)$ is

$$ddmin(c_{\text{✘}}) = ddmin_2(c_{\text{✘}}, 2) \quad \text{where}$$

$$ddmin_2(c'_{\text{✘}}, n) = \begin{cases} ddmin_2(\Delta_i, 2) & \text{if } \exists i \in \{1, \ldots, n\} \cdot test(\Delta_i) = \text{✘} \text{ (“reduce to subset”)} \\ ddmin_2(\nabla_i, \max(n-1, 2)) & \text{else if } \exists i \in \{1, \ldots, n\} \cdot test(\nabla_i) = \text{✘} \text{ (“reduce to complement”)} \\ ddmin_2(c'_{\text{✘}}, \min(|c'_{\text{✘}}|, 2n)) & \text{else if } n < |c'_{\text{✘}}| \text{ (“increase granularity”)} \\ c'_{\text{✘}} & \text{otherwise (“done”).} \end{cases}$$

where $\nabla_i = c'_{\text{✘}} - \Delta_i$, $c'_{\text{✘}} = \Delta_1 \cup \Delta_2 \cup \cdots \cup \Delta_n$, all Δ_i are pairwise disjoint, and $\forall \Delta_i \cdot |\Delta_i| \approx |c'_{\text{✘}}|/n$ holds.
The recursion invariant (and thus precondition) for $ddmin_2$ is $test(c'_{\text{✘}}) = \text{✘} \wedge n \leq |c'_{\text{✘}}|$.

Finding the global minimum would have an exponential complexity $(2^{|cX|})$, which is why Delta Debugging only gives us a local minimum: no subset of the reduced test case the test to fail, which is called 1-minimality.

- **Reducer Characteristics**: Advantages: Simplifies debugging (easier to debug, analyze reason), aids in identifying duplicate bugs, and improves communication of bug. Challenges: Interestingness tests can be complex; inefficient for highly structured languages without hierarchical reduction techniques.

Isolating Failure-Inducing Changes

- **Regression Bugs**: Features work in previous versions, fail in newer ones.
- **Git Bisect**: Automates the process of identifying the commit that introduced a regression bug using binary search.
  - Steps: Mark a known good version (`git bisect good`) and a bad version (`git bisect bad`). Test intermediate versions until the first bad commit is identified. Can be automated using scripts that build and test the program for each version.
- **Delta Debugging on Changes**: Applies delta debugging to individual patches within a commit. Granularities include hunks (consecutive differing lines) or entire changesets.

7. Software Development Practices

Development Practices: Architecture/SWE: 'What', Practices: 'How'. Between code and practices, need for agreements, no silver bullets, change over time, but there are generally good practices. What questions should we be asking? • Credits: Guest Lecturer Kareem Shehata.

Source Control (SCM)

Primary output of a SWE is code! But source control $\neq$ git.

- **Questions to ask**: Who is allowed to view / change code? How are changes managed? dependencies managed? How does source control integrate with your tools? Branches, tags, merges, etc.
- **SCM Tool Considerations**:
  - **Centralized vs Decentralized**: Centralized systems have single repository, decentralized allow multiple repositories w. synchronization.
  - **Dependency Management**: Tools (Maven, Gradle) help manage deps.
  - **Scalability**: Ensure SCM tool can handle large repositories and teams.
  - **Development Tool Support**: Integration with IDEs like VS, XCode.
  - **Expertise/opinions in team**: Choose tools that align with whole team's expertise, preferences.
- **Good SCM Practices**:
  - **Single Source of Truth**: Maintain single authoritative repository.
  - **Consistency**: Consistent naming conventions (branches, tags, commits).
  - **Small, Focused Changes**: Commit small, independent changes that are easier to review and revert if necessary.
  - **Clarity**: Clear commit messages explaining what was changed and why.
  - **Use Branches and Tags Appropriately**: Use branches for feature development and tags for releases.
  - **Avoid Long-Lived Branches**: Merge branches frequently to reduce integration challenges.
  - **Use Toolset for Managing Dependencies**: Leverage dependency management tools to handle external libraries.

Coding Style

Consistent coding style improves readability, maintainability, collaboration. Also, probably need to revisit, read own code in the future.

- **Why Care About Style?**:
  - Unnecessary changes in SCM due to formatting differences.
  - Inconsistencies make code harder to read and understand.
  - Differences in style can lead to confusion and inefficiency.
- **Key Elements of a Style Guide**:
  - **Consistency** $>>$ **Optimality**: Consistent code is more important than "clever" or overly optimized code.
  - **Focus on Clarity**: Code should be clear and unsurprising to readers.
  - **Practicality** $>>$ **Clever/Pretty**: Practical solutions are preferred over clever or aesthetically pleasing ones.
  - **Idioms and Practices**: Include guidelines on how tasks should be approached, not just formatting rules.
  - **Autoformatters**: Use tools like Prettier, clang-format, or Black to automate formatting.

Versioning and Releases

Depends on deploy method. Web services w. rolling releases, app stores, binaries, embed systems will be diff. in deployment, releases.

Software Versioning

Process of assigning unique identifiers to bundles of software. Identifiers help track, manage diff releases of software over time. Primary goals:

- Allow users to know which version of the software using, enable developers to identify version customers are using.
- Managing dependencies between software components.

Semantic Versioning (SemVer)

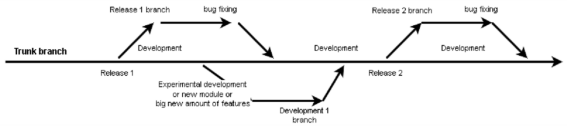
Semantic Versioning is a widely adopted versioning scheme, particularly for software with public APIs. It uses a three-part version number: **MAJOR.MINOR.PATCH**.

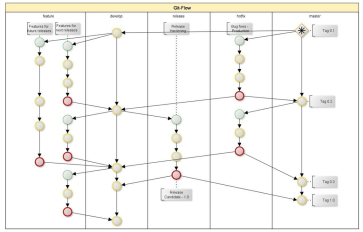
- **MAJOR version**: Increment: incompatible API changes are made.
- **MINOR version**: Increment: backward-compatible functionality is added.
- **PATCH version**: Increment: backward-compatible bug fixes.
- **Hyrum's Law**: With a sufficient number of users of an API, it does not matter what you promise in the contract: all observable behaviours of your system will be depended on by somebody.

This system ensures clarity and consistency in communicating changes to users and developers. However, these labels are slapped on by a human - no perfect guarantee it will not break your system!!

Versioning Schemes

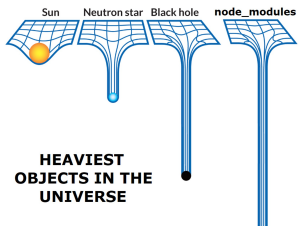
- **Commits as Identifiers**: Source control systems like Git assign unique hashes to commits (e.g., `2d54f09b`). **Pros**: Automatic, no additional effort required; useful for internal tracking. **Cons**: Not all commits correspond to release versions; difficult to remember and interpret.
- **Assigned Identifiers**: Includes code names (e.g., Android's "Lollipop," "Marshmallow") or version increments (e.g., v1, v2, v3). Can be combined with tags indicating release stages (e.g., alpha, beta, rc for release candidate). **Pros**: Easier to remember; code names aid branding and communication. **Cons**: Requires manual assignment and management.
- **E.g. Versioning in Practice**: **Android**: Uses both numeric versions and dessert-themed code names (e.g., Android 9: Pie). Code names follow alphabetical order, aiding in identifying newer versions. **macOS**: Combines numeric versions with thematic names (e.g., macOS Big Sur, macOS Monterey). **Ubuntu**: Uses calendar-based versioning (YY.MM) and distinguishes Long-Term Support (LTS) releases.
- **Choosing Versioning Scheme**: The choice of versioning scheme depends on organizational needs, deployment strategies, release frequency, and user expectations. For example: web services with rolling releases may prefer frequent, small increments. Embedded systems/ safety-critical software may strict versioning/ long-term support. (Needs of marketing / users?)
- **Best branching strategy?**: Anything that gives a single source of truth, consistent, depends on your organization, deployment strategy, etc!

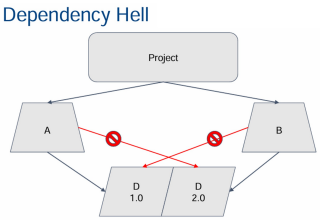




Dependencies

Probably most important, most difficult part of SWE. Dependencies: external components for software to function correctly, but not directly controlled by development team. Modern software: lot of dependencies!





- **Examples**: Libraries (e.g., React, TensorFlow), Build tools (e.g., Maven, Gradle), External services (e.g., APIs, cloud platforms).
- **Dependency Hell**: when multiple dependencies conflict, when outdated libraries cause integration issues. Modern software numerous dependencies, increasing complexity of dependency management.
- **Managing Dependencies** Strategies include: Embedding small or critical libraries directly into project, hosting medium-sized libraries in a private repository, using submodules or package managers for large, well-established libraries. Employing lockfiles and containers to ensure consistent runtime environments.

Deprecation

Process of orderly migration away from, eventual removal of obsolete systems. Planning, executing a transition strategy to minimize disruption.

- **Why Deprecate?** Code / Dependencies are liability, not an asset. Maintaining outdated systems incurs operational costs. Legacy systems hinder the evolution of newer systems due to compatibility requirements. Deprecation allows organizations to focus resources, maintain and improve current systems. (Google: having multiple systems performing same function impedes evolution of newer system as still expected to maintain compatibility with old one).
- **Challenges**: Emotional attach. old systems. Difficulty in funding, executing deprecation efforts. High costs migrating to entirely new systems.
- **Types of Deprecation**: **Advisory Dep**: Informing users about deprecated features without enforcing immediate action. **Compulsory Dep**: Setting deadlines for removal and ensuring compliance through enforcement.
- **Deprecation During Design**: Designing systems with deprecation in mind can simplify future decommissioning. For example: Planning for incremental replacement of components. Ensuring clear documentation and migration paths for users (give alternative).

Code Review

Before deploying new code, critical to ensure quality, correctness. Unreviewed code: lead to bugs, poor design decisions, maintainability issues. Mitigate risks by involving others in process of inspecting, improving code.

- **Risk Mitigation**: Code reviews catch defects early, formal reviews identifying up to 60% of defects (comparatively - 30% for testing).
- **Maintainability**: Studies: up to 75% of code review comments address evolvability, maintainability, not just functionality.
- **Other Benefits**: Solicit feedback on code design, practices. Teaching, demonstrating coding methods. Raising awareness of product's state. Highlighting potential future problems.
- **Challenges**: *Speed vs. Thoroughness*: Review time increases exponentially with the size of the change, leading to diminishing returns on thoroughness. *Balancing Priorities*: if speed or depth of review more important.

Good Practices for Code Reviews

To ensure effective code reviews, **authors** should:

- Keep reviews small and focused. Clear desc, annotate to guide reviewers.
- Choose appropriate reviewers, balance feedback quality w. team awareness.
- Open to feedback, implement suggested changes unless strong reason not.
- Reflect on feedback to improve future work.

Reviewers should:

- Push back on overly broad or poorly described changes.
- Focus on code, not person. Ask qns, point out problems, suggest solutions.
- Consider edge cases, failure scenarios, and future maintainability.
- Be responsive while balancing the severity and importance of changes.

Software Quality Assurance (SQA)

Software Quality Assurance (SQA) encompasses all activities aimed at ensuring product quality before reach customers. Unlike testing, which focuses on finding bugs, SQA addresses broader aspects of quality, including usability, performance, and maintainability.

- **Key Principle**: Just as physical products quality checks, software products to meet high-quality standards before release.
- **SQA Scope**: test, docs, process adherence, customer UX validation.
- **Best Practices**: Simulate real-world customer usage to identify gaps between product, expectations. Ensure QA performed by someone other than developer to avoid biases. Focus on entire customer experience, not just crashes, functional bugs. (E.g. QA engineer tests edge cases (e.g., ordering -1 beers) and unexpected inputs (e.g., "ueicbksjdhd","where is toilet").)

Deployment

Deployment strategy impacts SWE practices. Method of deployment depends on factors e.g. update frequency, scalability, risk tolerance.

- **Examples of Deployment Methods**: Zip files on a server, Web apps w. continuous deployment ("DevOps"). App stores (mobile), Physical media (e.g. games), Pre-installed software on devices.
- **Considerations**: Speed, friction for releases/updates. Scalability to handle large user bases. Risks associated with deployment failures. Costs of maintaining infrastructure.
- **Deployment Strategy Examples**: *NASA*: Rigorous testing, version control, minimal runtime dependencies due to limited update opportunities. *Web App*: Allows instant updates, frequent releases, enabling rapid iteration.

Key Takeaways

- Learn best practices but adapt them to your organization's needs.
- There are no universal solutions; context matters.
- Teamwork, collaboration, and consistency outperform individual heroics.
- Think critically about organizational priorities when choosing practices.

Differences Between Junior and Senior Developers

- **Junior Devs**: Hide code until "ready." Provide inaccurate estimates, deliver inconsistently. Use long-lived branches, big merges. Large, unstructured commits w. unclear messages. View code review feedback as criticism.
- **Senior Devs**: Share code, ideas during dev. Provide accurate estimates, deliver on time. Use short-lived branches w. smaller merges. Make small, focused commits w. clear reasoning. View code review feedback as opportunity to improve.
- **Agility in Software Engineering**: "Solutions to common software engineering problems not always obvious. Agility - both technical and organizational - is essential to identify solutions that work for current challenges and remain resilient to future changes." - Asim Husain, VP Eng, Google.

8. Static Analysis and Type System

Limitations of Testing

- **Testing** widely used to identify bugs, but has significant limitations. Demonstrates presence of bugs but cannot guarantee their absence. Often time-consuming and resource-intensive. Edge cases and rare conditions may remain untested.
- **Historical Context of Software Failures**: significant financial losses and even endangerment of human lives. E.g. Mars Climate Orbiter (1998), Knight Capital Group (2012) trading glitch loss $440 million. Wannacry Ransomware Attack (2017). Boeing 737 MAX Crisis (2019).
- **Static analysis, type systems** can help play a crucial role in improving software reliability and reducing the incidence of costly bugs (Fewer runtime errors).

Static Analysis

- **Static analysis**: examination of code without executing it.
- **Primary goals**: identifying potential errors, adherence to coding standards, optimizing performance.
- **Why Static Analysis**: Early detection of bugs, reduce fix cost later in dev process. Improved code quality by enforcing best practices. Reduced debugging time through automated checks. Enhanced maintainability by ensuring consistent coding styles and fewer runtime errors.
- **Types of Static Analysis**:
  - **Linters and Style Checkers**: Focus on formatting and adherence to coding conventions without deeper semantic analysis.
  - **Data-Flow Analysis**: Tracks the flow of data through a program to determine possible values at different points in the execution.
  - **Control-Flow Analysis**: Analyze order of statement/function execution.
  - **Type Checking**: Verifies that variables and expressions are used consistently according to their declared types. (Focus on this)

The Problem with Unsafe Languages: E.g. Python, C, C++, and JavaScript, are inherently unsafe. These languages attempt to execute programs that may fail at runtime due to type mismatches or other errors.

- Python evaluates unsafe programs, throw runtime error when issues arise.
- C and C++ place the burden of writing safe programs entirely on the programmer, often leading to manual errors.
- Relying on manual approach to avoid errors proven unsuccessful.

Type System, Role in Static Analysis

- **A type system**: tractable syntactic method for proving absence of certain prog. behaviors by classifying phrases according to kinds of values they compute. (Benjamin Pierce, Types and Programming Languages)
- Type systems serve as critical component of static analysis, detect type-related errors before runtime. A type system acts as a filter.
- **Well-Typed Programs**: Adhere to type rules, allowed to execute. (e.g. $0 + 1$)
- **Ill-Typed Programs**: Prog w. Type mismatch reject before execution (e.g. $false\ 0$ - cannot apply boolean to integer).
- **Benefits of Type Systems**:
  - **Safety**: Prevents runtime errors caused by type mismatches.
  - **Early Error Detection**: Identifies errors during the development phase rather than at runtime.
  - **Improved Readability and Maintainability**: Clear type annotations make code easier to understand and modify.

λ-Calculus

- **Prog. language** invented in 1930s by Alonzo Church and Stephen Cole Kleene.
- "Core language" w. essential features to express all computation. (Lambda calculus is Turing complete). No redundancy, encode extra features as 'syntactic sugar.'
- Foundations of functional programming (e.g., Lisp, ML, Haskell).
- Often used as a core language to study language theories. (Type systems, Scope and binding, Higher-order functions, Denotational semantics, Program equivalence)

Overview: λ-Calculus as a Language

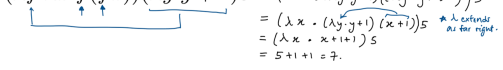
- **Syntax** (How to write program): Keyword "λ" for defining functions.
- **Semantics** (How to describe execution): *reduction* as calculation rules.
- Type system, Model theory (not covered)

Syntax

- λ-terms or λ-expressions: (Terms) $M, N ::= x \mid \lambda x.M \mid MN$
- Lambda abstraction ($\lambda x.M$): Anonymous functions.
- Lambda application (MN). Examples:
  - `int f(int x) { return x; }` $\Leftrightarrow \lambda x.x$
  - `f(3);` $\Leftrightarrow (\lambda x.x)3$
- **Pure/Extended λ-Calculus**: Pure λ-calculus only has λ-terms. Extended has added extra operations and data types. (E.g. $\lambda x.(x+1)$, $\lambda z.(x+2y+z)$)
- **Conventions (Associativity)**
  - **Body of λ extends as far to the right as possible**: $\lambda x.MN$ means $\lambda x.(MN)$, not $(\lambda x.M)N$
  - **Function applications are left-associative**: MNP means $(MN)P$, not $M(NP)$
  - Examples:
    $(\lambda x.\lambda y.x - y)53 \quad = \quad ((\lambda x.\lambda y.x - y)5)3$
    $(\lambda f.\lambda x.fx)(\lambda x.x+1)2 \quad = \quad ((\lambda f.\lambda x.fx)(\lambda x.x+1))2$

Higher-Order Functions

- Functions as **return values**: $\lambda x.\lambda y.x - y$
- Functions as **arguments**: $(\lambda f.\lambda x.fx)(\lambda x.x+1)2$
- **Example**: Function Composition:
  Given function f, return function $f \circ f$: $\lambda f.\lambda x.f(fx)$



Curried Functions

- λ abstraction is a function of one parameter.
- $\lambda x.\lambda y.x - y \quad \leftrightarrow$ `int f(int x, int y) { return x - y; }`: Computationally equivalent (can be transformed into each other).
- **Curry**: Transform $(\lambda(x,y).x-y)$ to $(\lambda x.\lambda y.x-y)$. Uncurry: Reverse of Curry.

Free and Bound Variables

- E.g. $\lambda x.x + y$. x: Bound variable. y: Free variable.
- Bound variables renameable (α-equivalence): $\lambda x.(x+y) \Leftrightarrow \lambda z.(z+y)$
- Name of free variables matters: $\lambda x.(x+y) \neq \lambda x.(x+z)$
- **Occurrences**: $(\lambda x.x + y)(x+1) : x$ has both free and bound occurrence.

Processes in Lambda Calculus

1. **Alpha Conversion**: If are applying two lambda expressions w. same variable name inside, change one of them to a new variable name. E.g. $(\lambda x.xx)(\lambda x.x) \rightarrow (\lambda x.xx)(\lambda y.y)$
2. **Beta Reduction**: Basically just substitution. Process of calling lambda expression with input, and getting output. E.g. $(\lambda x.xy)z \implies zy$. Substitution: $(\lambda x.M)N \rightarrow M[N/x]$. ('N replaces x in M'). Repeatedly apply the reduction rule to any sub-term.
3. **Eta Conversion/Eta Reduction**: Special case reduction. Converts between $\lambda x.(fx)$ and f whenever x does not appear free in f. Basically means $\lambda x.(fx) = f$ if f does not make use of x. E.g. $(\lambda x.(\lambda y.yy)x) = (\lambda y.yy)$

Formal Definitions

Terms:

$$M, N ::= x \mid \lambda x.M \mid MN$$

- **Set of free variables** in M: $fv(M)$

  $fv(x) \coloneqq \{x\}, \quad fv(\lambda x.M) \coloneqq fv(M) \backslash \{x\}, \quad fv(MN) \coloneqq fv(M) \cup fv(N)$

  E.g.: $fv((\lambda x.x)x) = \{x\}, \quad fv((\lambda x.x+y)x) = \{x, y\}$
- **Substitution**:

  $M[N/x] : N$ replaces x in M

  - Defined by induction on terms:

  $$x[N/x] \coloneqq N, \quad y[N/x] \coloneqq y \quad (\text{if } y \neq x)$$
  $$(MP)[N/x] \coloneqq (M[N/x])(P[N/x])$$
  $$(\lambda x.M)[N/x] \coloneqq \lambda x.M \quad (\text{Only replace free variables!})$$

  Substitution – Avoid Name Capture. Rename bound variables before substitution:

  $$(\lambda x.x - y)[x/y] = (\lambda z.z - y)[x/y] = \lambda z.z - x$$
- **Alpha-equivalence**: $\lambda x.M = \lambda y.M[y/x]$, where y is fresh.. Two terms are α-equivalent iff one convertable to other purely by renaming bound variables.
- **Beta-reduction** (substitution): $(\lambda x.M)N \rightarrow M[N/x]$. ('N replaces x in M'). Repeatedly apply the reduction rule to any sub-term.

Normal Form

- **Reducible Expression** (β-redex): A term of the form $(\lambda x.M)N$.
- β-normal form: A term containing no β-redex.
- Stopping point: Cannot further apply β-reduction rules.
  E.g. $(\lambda f.\lambda x.f(fx))(\lambda y.y+1)2 \rightarrow 2+1+1$ (β-normal form)
  Can further reduce to 4 if reduction rules for $+$ are defined.
- E.g.: $\lambda x.\lambda y.x$ (normal), $(\lambda x.\lambda y.x)(\lambda z.z)$ and $\lambda x.(\lambda y.x)(\lambda z.z)$ (not normal)

Confluence (Church-Rosser Property)

- Terms can be evaluated in any order. Final result (if exists) is uniquely determined.
- 'When applying reduction rules to terms, the ordering in which the reductions are chosen does not make a difference to the eventual result.'
- **Confluence Theorem**: For all M, M_1, M_2, if $M \rightarrow^* M_1$ and $M \rightarrow^* M_2$, then $\exists M'$ such that $M_1 \rightarrow^* M'$ and $M_2 \rightarrow^* M'$.

Non-Terminating Reduction

- Some terms have no normal forms.
- E.g. $(\lambda x.xx)(\lambda x.xx) \rightarrow (\lambda x.xx)(\lambda x.xx) \rightarrow \ldots$
- Term may have both terminating and non terminating reduction sequences. E.g.

$$(\lambda u.\lambda v.v)((\lambda x.xx)(\lambda x.xx))$$
$$\rightarrow \lambda v.v \quad (\text{Terminating})$$
$$\rightarrow (\lambda u.\lambda v.v)((\lambda x.xx)(\lambda x.xx)) \rightarrow \ldots \quad (\text{Non-terminating})$$

Reduction Strategies

- **Normal-order reduction**: Choose the left-most, outer-most redex first.

$$(\lambda u.\lambda v.v)((\lambda x.xx)(\lambda x.xx)) \to \lambda v.v$$

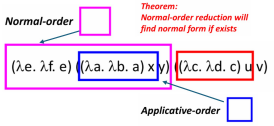
- Normal-order reduction will find a normal form if it exists.
- **Applicative-order reduction**: Choose the left-most, inner-most redex first.

$$(\lambda u.\lambda v.v)((\lambda x.xx)(\lambda x.xx)) \to (\lambda u.\lambda v.v)((\lambda x.xx)(\lambda x.xx)) \to \dots$$

Reduction strategies – examples

Normal-order	***Applicative-order***
(λx. x x) ((λ.y. y) (λ.z. z))	(λx. x x) ((λ.y. y) (λ.z. z))
→ ((λ.y. y) (λ.z. z)) ((λ.y. y) (λ.z. z))	→ (λx. x x) (λ.z. z)
→ (λ.z. z) ((λ.y. y) (λ.z. z))	→ (λ.z. z) (λ.z. z)
→ (λ.y. y) (λ.z. z)	→ λ.z. z
→ λ.z. z	

Applicative-order may ***not*** be as efficient as normal-order when the argument is not used.



Programming in λ-Calculus

- **Encoding Boolean Values and Logical Operators**

$$\text{True} := \lambda x.\lambda y.x, \quad \text{False} := \lambda x.\lambda y.y$$

$$\text{Negation: not} := \lambda b.b\,\text{False}\,\text{True}$$

$$\text{Conjunction: and} := \lambda b.\lambda b'.b\,b'\,\text{False}$$

$$\text{Disjunction: or} := \lambda b.\lambda b'.b\,\text{True}\,b'$$

$$\text{Conditional: if b then M else N} := b\,M\,N$$

$$\text{Alternative negation: not'} := \lambda b.\lambda x.\lambda y.b\,y\,x$$

- **Church Numerals**: - Encoding natural numbers:

$$0 := \lambda f.\lambda x.x, \quad 1 := \lambda f.\lambda x.f\,x, \quad 2 := \lambda f.\lambda x.f\,(f\,x)$$

$$\text{General form: } n := \lambda f.\lambda x.f^n\,x$$

$$\text{Successor function: succ} := \lambda n.\lambda f.\lambda x.f\,(n\,f\,x)$$

$$\text{Example: succ n} \to \lambda f.\lambda x.f\,(n\,f\,x) = \lambda f.\lambda x.f^{n+1}\,x = n+1$$

$$\text{Zero-check function: iszero} := \lambda n.\lambda x.\lambda y.n\,(\lambda z.y)\,x$$

$$\text{Addition: add} := \lambda n.\lambda m.\lambda f.\lambda x.n\,f\,(m\,f\,x)$$

$$\text{Multiplication: mult} := \lambda n.\lambda m.\lambda f.n\,(m\,f)$$

- **Pairs and Tuples**:

$$\text{Pair encoding: } (M,N) := \lambda f.f\,M\,N$$

$$\text{Projection functions:} \pi_0 := \lambda p.p\,(\lambda x.\lambda y.x), \quad \pi_1 := \lambda p.p\,(\lambda x.\lambda y.y)$$

$$\text{Tuple encoding: } (M_1,\dots,M_n) := \lambda f.f\,M_1\,\dots\,M_n$$

$$\text{Projection functions:} \pi_i := \lambda p.p\,(\lambda x_1.\dots.\lambda x_n.x_i)$$

- **Lists**: can be encoded using pairs. Operations on lists (e.g., head, tail, length) can be defined similarly. **Others**: Trees, Recursive functions etc.
- **Main Points About λ-Calculus**: Succinct function expressions using λ.Bound variables can be renamed (α-equivalence). Reduction via substitution (β-reduction). Can be extended with: Types (e.g., simply-typed λ-calculus), and side-effects (not covered here).

untyped λ-calculus

- Syntax: notation for defining functions

(Terms) M, N ::= x | λx. M | M N

- Semantics: reduction rules

$$\frac{}{(\lambda x.M)N \;\to\; M[N/x]}\ (\beta)$$

$$\frac{M \to M'}{M\,N \to M'N} \qquad \frac{N \to N'}{M\,N \to M\,N'} \qquad \frac{M \to M'}{\lambda x.M \to \lambda x.M'}$$

Free and bound variables

- fv(M): the set of free variables in M

fv(x) ≝ {x}

fv(λx.M) ≝ fv(M) \ {x}

fv(M N) ≝ fv(M) ∪ fv(N)

Typed Lambda Calculus

- Untyped λ-calculus is powerful but lacks safety guarantees.
- Add type system ensures correctn, prevent certain errors (e.g., apply non-functions).
- Provides formal framework for reasoning about programs. **type safety**:'Well-typed programs cannot go wrong.' Enables static analysis and optimization.
- **Type System**: Typing rules (assign types to terms), and Type safety (soundness of rules). Well-typed terms cannot go wrong.

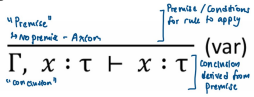
Simply-Typed Lambda Calculus (STLC)

- Extends untyped λ-calculus with a simple type system. **Syntax**:

$$\text{(Types)} \quad \tau,\sigma ::= T \mid \sigma \to \tau$$

$$\text{(Terms)} \quad M,N ::= x \mid \lambda x:\tau.M \mid M\,N$$

- Functions classified using argument and result types. $(\sigma \to \tau)$
- $\to$ is right-associative: $\tau \to \tau \to \tau$ is $\tau \to (\tau \to \tau)$
- **Typing Context** (Γ): a set of typing assumptions (to assign types to terms):
  - $[\Gamma \vdash M : \tau]$ means 'Under typing context Γ, term M is well-typed, has type τ.'
  - Typing context $[\,\Gamma \;::= \cdot \mid \Gamma, x:\tau\,]$ (Empty context + extended context)
  - $\vdash$ (pronounced 'turnstile'), also means 'proves / entails'. (e.g. Γ proves $x:\tau$).
- **Typing (inference) rules** (to derive the typing judgement):
  - *Variable rule*: (x is type τ under context where it is defined as τ)



  - *Application rule*: (Type of fn. application determined by type of fn, and param).

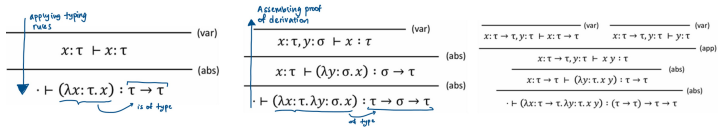
$$\frac{\Gamma \vdash M : T_1 \to T_2 \quad \Gamma \vdash N : T_1}{\Gamma \vdash M\,N : T_2}$$

  - *Abstraction rule*:

$$\frac{\Gamma, x : T_1 \vdash M : T_2}{\Gamma \vdash \lambda x : T_1.M : T_1 \to T_2}$$

- **Typing Judgement**: a formal statement in type theory that asserts that a term (or expression) in a programming language has a specific type, under a given set of assumptions about the types of variables.

Typing derivation examples



Soundness and Completeness

- **Soundness**: A sound type system never accepts a program that can 'go wrong.' No false negatives: The language is type-safe. Ensures execution never reaches a "meaningless" state.
- **Completeness**: A complete type system never rejects a program that cannot go wrong. No false positives.
- For any Turing-complete programming language, the set of programs that may "go wrong" is undecidable. Type systems cannot be both sound and complete.
- In practice, we prioritize soundness and try to minimize false positives.

Soundness in STLC

- **Type Safety Theorem**: (Reduction of a well-typed term either diverges, or terminates in a value of the expected type.) Follows from two key lemmas.

$$\text{For any } M, M' \text{ and } \tau, \text{ if } \cdot \vdash M : \tau \text{ and } M \to^* M', \text{ then:}$$

$$\cdot \vdash M' : \tau, \quad \text{and either } M' \in \text{Values or } \exists M''.M' \to M''.$$

- **Preservation (Subject Reduction)**: well-typed terms reduce only to well-typed terms of same type. ($\to$ means reduces to)

$$\text{If } \cdot \vdash M : \tau \text{ and } M \to M', \text{ then } \cdot \vdash M' : \tau.$$

- **Progress**: a well-typed term is either a value or can be reduced.

$$\text{If } \cdot \vdash M : \tau, \text{ then either } M \in \text{Values or } \exists M'.M \to M'.$$

Properties of STLC

- **Strong Normalization Theorem**: Well-typed terms in STLC always terminate. E.g. $(\lambda x.xx)(\lambda x.xx)$ cannot be assigned a type.
- **Limitation: Not Complete**: Some valid programs may be rejected by type system.

$$(\lambda x.(x(\lambda y.y))(x3))(\lambda z.z)$$

- Cannot find types σ, τ such that: $[x : \sigma \vdash (x(\lambda y.y))(x3) : \tau]$
- However, reduces successfully: $[((\lambda z.z)(\lambda y.y))((\lambda z.z)3) \to (\lambda y.y)3 \to 3]$

Simply-typed λ-calculus (STLC)

(Types) τ, σ ::= T | σ → τ

(Terms) M, N ::= x | λx : τ. M | M N

Typing rules

$$\frac{}{\Gamma,\ x:\tau \vdash x:\tau}\ (\text{var}) \qquad \frac{\Gamma, x:\sigma \vdash M:\tau}{\Gamma \vdash (\lambda x{:}\sigma.M) : \sigma \to \tau}\ (\text{abs})$$

$$\frac{\Gamma \vdash M : \sigma \to \tau \qquad \Gamma \vdash N : \sigma}{\Gamma \vdash M\,N : \tau}\ (\text{app})$$

Soundness (type safety)

Reduction rules

$$\frac{}{(\lambda x{:}\tau.M)N \;\to\; M[N/x]}\ (\beta)$$

$$\frac{M \to M'}{M\,N \to M'N} \qquad \frac{N \to N'}{M\,N \to M\,N'} \qquad \frac{M \to M'}{\lambda x{:}\tau.M \to \lambda x{:}\tau.M'}$$

Extending STLC

Use STLC as a foundation for understanding other common language constructs.

- Extend the syntax (types & terms), operational semantics (reduction rules)
- Extend the type system (typing rules), soundness proof (new proof cases)
- **Adding Product Types** ($\sigma \times \tau$):
  - Represent structured data, combining two values into a pair. Logical intuition: To make a $\sigma \times \tau$, we need both a σ and a τ. Operations: Construct pairs, Extract components using projections, E.g.: Swapping components of a pair.
- **Sum Types** ($\sigma + \tau$):
  - Represent choices between alternatives. Logical intuition: To make a $\sigma + \tau$, we need either a σ or a τ. Operations: Inject values into the sum type (left or right). Handle alternatives using case analysis. Example: Encoding booleans as true + false.
- Extended STLC includes functions, products, and sums.
- Products ($\sigma \times \tau$) and sums ($\sigma + \tau$) are logical duals.
- These constructs model common programming patterns like structs, unions, and enums.